



Universidad
Rafael Landívar
Identidad Jesuita en Guatemala

Universidad Rafael Landívar

Facultad de Ingeniería

Ingeniería en Informática y Sistemas

Manejo e Implementación de Archivos

Catedrático: Ingeniero Luis Enrique

Laboratorio 13 – APIs con Swagger y Postman

Diego Alessandro Lopez Higueros – 1187525

10/2/2026



Nombre: Diego Alessandro Lopez Higueros **Carné:** 1187525

Objetivo: Comprender y consumir una API REST mediante Swagger UI y Postman, identificando endpoints, métodos HTTP, parámetros, cuerpos JSON, códigos de respuesta y operaciones CRUD. El estudiante deberá interpretar la documentación de una API, ejecutar solicitudes y resolver un reto práctico utilizando evidencia técnica.

Investigación previa:

1. ¿Qué es una API REST?

Es una interfaz de programación que habilita la comunicación entre dos sistemas mediante el protocolo HTTP, bajo los principios de la arquitectura REST. En este modelo, la información se organiza en recursos identificables mediante URLs únicas que se manipulan utilizando los métodos HTTP estándar.

2. ¿Qué es un endpoint?

Es la ruta de acceso específica de una API diseñada para ejecutar una acción sobre un recurso. Surge de la combinación de una dirección URL y un método HTTP determinado.

3. ¿Qué diferencia existe entre una URL y un endpoint?

La URL es la dirección web que ubica a un recurso de forma global (por ejemplo, https://miapp.com/api/usuarios/5). El endpoint define la intersección entre esa URL y una acción HTTP específica. Por ende, una única URL puede representar múltiples endpoints según el verbo HTTP que se utilice (como GET o DELETE).

4. ¿Qué función cumplen los métodos HTTP GET, POST, PUT/PATCH y DELETE?

- GET: Consulta y recupera información sin alterar el estado del servidor.
- POST: Envía datos para crear un nuevo recurso en el sistema.
- PUT: Sustituye un recurso existente en su totalidad.
- PATCH: Aplica modificaciones parciales sobre un recurso.
- DELETE: Remueve o elimina un recurso determinado.

5. ¿Qué es JSON y por qué es común en las APIs?

JSON (*JavaScript Object Notation*) es un formato liviano de intercambio de datos estructurado en pares clave-valor y listas. Se utiliza masivamente en APIs por ser fácil de interpretar tanto por humanos como por máquinas, consumir poco ancho de banda y ser nativamente compatible con la mayoría de los lenguajes de programación.

6. ¿Qué es un código de estado HTTP? Mencione ejemplos de 200, 404 y 500.

Es una clave numérica de tres dígitos enviada por el servidor para indicar el resultado de una solicitud HTTP.



- 200 OK: La petición se procesó exitosamente.
- 404 NOT FOUND: El recurso solicitado no existe en el servidor.
- 500 INTERNAL SERVER ERROR: Ocurrió un fallo imprevisto dentro del servidor al procesar la solicitud.

7. ¿Qué información se puede encontrar en la respuesta de una API?

- Código de estado: Notifica si la operación concluyó con éxito o falló.
- Encabezados (*Headers*): Metadatos clave como el formato del contenido (Content-Type), políticas de caché o credenciales de autenticación.
- Cuerpo (*Body*): La carga útil con los datos solicitados (normalmente en JSON) o el detalle del error en caso de fallo.
- Métricas de rendimiento: El tiempo de procesamiento y el tamaño de la respuesta (visibles en clientes como Postman).

8. ¿Qué ventajas tiene utilizar Postman para probar una API antes de programarla?

- Permite validar el funcionamiento de los endpoints sin necesidad de desarrollar la interfaz o el cliente.
- Facilita la definición y ajuste de las estructuras de datos de la API de forma temprana.
- Ayuda a detectar fallos de lógica, rutas o parámetros antes de escribir el código final.
- Permite organizar las peticiones en colecciones reutilizables para compartir con el equipo de trabajo.

9. ¿Qué es y para qué sirve Swagger UI?

Es un entorno visual que convierte especificaciones OpenAPI en documentación web interactiva. Sirve para visualizar de forma clara todos los endpoints, sus parámetros esperados y las respuestas posibles, permitiendo además realizar pruebas en tiempo real desde el navegador mediante la función "Try it out".

10. ¿Qué función cumple Postman en el desarrollo y prueba de APIs?

Postman actúa como una plataforma integral para el diseño, depuración, documentación y automatización de APIs. Su función principal consiste en enviar solicitudes HTTP al servidor, permitir la inspección exhaustiva de las respuestas recibidas y certificar que la API se comporte según lo planificado.



1. Preparación del entorno

Instalar o utilizar Postman: <https://www.postman.com/downloads/>

API a explorar, Swagger Petstore 3: <https://petstore3.swagger.io/>

Importante: La actividad se realizará mediante solicitudes HTTP GET y análisis de las respuestas JSON. No se requiere API Key para esta práctica.

2. Exploración inicial

Ingresa a Swagger Petstore y localice las operaciones relacionadas con /pet

- Identifique un endpoint que permita consultar una mascota por ID. **/pet/{petId}**
- Indique el método HTTP utilizado. **GET**
- Indique qué parámetro necesita la solicitud. **petId**
- Ejecute la solicitud con Try it out y registre el código HTTP recibido. **200 OK si existe la mascota o 404 not found sino.**
- Explique con sus palabras qué información contiene la respuesta. **Incluye datos estructurados con parámetros de la mascota, desde el id, nombre, hasta el estado de su disponibilidad.**

Evidencia: captura de Swagger mostrando la operación y su respuesta.

The screenshot displays the Swagger UI for the Petstore 3 API. The 'Parameters' section shows a required parameter 'petId' of type 'integer' with a value of '10'. Below this, the 'Execute' button is highlighted. The 'Responses' section shows the response for the '200' status code, which is a JSON object containing pet details: { "id": 10, "name": "Borja", "photoUrls": [], "tags": [] }. The 'Curl' section shows the command: curl -X GET -H 'accept: application/json' https://petstore3.swagger.io/api/v3/pet/10. The 'Request URL' is also shown as https://petstore3.swagger.io/api/v3/pet/10.



Response headers

```
access-control-allow-headers: Content-Type,api_key,authorization
access-control-allow-methods: GET,POST,DELETE,PUT
access-control-allow-origins: *
access-control-expose-headers: Content-Disposition
content-length: 62
content-type: application/json
date: Sat, 01 Oct 2020 01:35:56 GMT
server: Jetty(9.4.53.v20210809)
```

Responses

Code	Description	Links
200	successful operation	No links
400	Invalid ID supplied	No links
404	Pet not found	No links
default	Unexpected error	No links

Media type

application/json

Controls Accept header

Example Value Schema

```
{
  "id": 10,
  "name": "Doggie",
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "available"
}
```

3. Crear un recurso con POST

Utilice POST /pet para crear una mascota. Construya un JSON propio; no copie exactamente el ejemplo de Swagger.

El JSON debe incluir, como mínimo:

id
name
status
category
tags

Registre el ID utilizado y verifique la respuesta. Luego consulte nuevamente la mascota por medio de GET /pet/{petId}.

Evidencia: JSON enviado + respuesta de creación + consulta posterior.

JSON enviado:

```
{
  "id": 1187525,
  "category": {
    "id": 10,
    "name": "Caninos"
  },
  "name": "Rocco",
  "photoUrls": [
    "https://example.com/rocco.jpg"
  ],
  "tags": [
    {
      "id": 1,
      "name": "entrenado"
    }
  ]
}
```



```
}  
},  
"status": "available"  
}
```

Respuesta de Creación:

Code

Details

200

Response body

```
{  
  "id": 1187525,  
  "category": {  
    "id": 10,  
    "name": "Caninos"  
  },  
  "name": "Rocco",  
  "photoUrls": [  
    "https://example.com/rocco.jpg"  
  ],  
  "tags": [  
    {  
      "id": 1,  
      "name": "entrenado"  
    }  
  ],  
  "status": "available"  
}
```

Download

Response headers

```
access-control-allow-headers: Content-Type,api_key,Authorization  
access-control-allow-methods: GET,POST,DELETE,PUT  
access-control-allow-origin: *  
access-control-expose-headers: Content-Disposition  
content-length: 171  
content-type: application/json  
date: Sat,03 Oct 2026 01:46:14 GMT  
server: Jetty(9.4.53.v20231009)
```

Consulta posterior:

Code

Details

200

Response body

```
{  
  "id": 1187525,  
  "category": {  
    "id": 10,  
    "name": "Caninos"  
  },  
  "name": "Rocco",  
  "photoUrls": [  
    "https://example.com/rocco.jpg"  
  ],  
  "tags": [  
    {  
      "id": 1,  
      "name": "entrenado"  
    }  
  ],  
  "status": "available"  
}
```

Download

Response headers

```
access-control-allow-headers: Content-Type,api_key,Authorization  
access-control-allow-methods: GET,POST,DELETE,PUT  
access-control-allow-origin: *  
access-control-expose-headers: Content-Disposition  
content-length: 171  
content-type: application/json  
date: Sat,03 Oct 2026 01:53:54 GMT  
server: Jetty(9.4.53.v20231009)
```



4. Modificar el recurso

- Modifique la mascota creada utilizando PUT /pet. Cambie al menos dos atributos, por ejemplo nombre y estado.
- ¿Qué método HTTP utilizó y por qué? **R/ PUT, en API REST se utiliza para actualizar o reemplazar completamente la representación de un recurso existente.**
- ¿Qué datos fueron enviados en el body? **R/ Se envió el JSON completo con la información de la mascota, manteniendo el ID original y modificando los atributos de nombre y estado.**
- ¿Qué código HTTP devolvió la API? **R/ 200 OK**
- ¿Cómo comprobó que el cambio realmente ocurrió? **R/ Se ejecutó nuevamente una consulta GET y al revisar el cuerpo de la respuesta se verificó que en efecto, los atributos fueron modificados.**

Nuevo JSON:

```
{
  "id": 1187525,
  "category": {
    "id": 10,
    "name": "Caninos"
  },
  "name": "Jake",
  "photoUrls": [
    "https://example.com/rocco.jpg"
  ],
  "tags": [
    {
      "id": 1,
      "name": "entrenado"
    }
  ],
  "status": "sold"
}
```

Resultado:



Code

Details

200

Response body

```
{
  "id": 1187525,
  "category": {
    "id": 10,
    "name": "Caninos"
  },
  "name": "Jake",
  "photoUrls": [
    "https://example.com/rocco.jpg"
  ],
  "tags": [
    {
      "id": 1,
      "name": "entrenado"
    }
  ],
  "status": "sold"
}
```

Download

Response headers

```
access-control-allow-headers: Content-Type,api_key,Authorization
access-control-allow-methods: GET,POST,DELETE,PUT
access-control-allow-origin: *
access-control-expose-headers: Content-Disposition
content-length: 165
content-type: application/json
date: Sat,03 Oct 2026 01:56:01 GMT
server: Jetty(9.4.53.v20231009)
```

GET Posterior:

Code

Details

200

Response body

```
{
  "id": 1187525,
  "category": {
    "id": 10,
    "name": "Caninos"
  },
  "name": "Jake",
  "photoUrls": [
    "https://example.com/rocco.jpg"
  ],
  "tags": [
    {
      "id": 1,
      "name": "entrenado"
    }
  ],
  "status": "sold"
}
```

Download

Response headers

```
access-control-allow-headers: Content-Type,api_key,Authorization
access-control-allow-methods: GET,POST,DELETE,PUT
access-control-allow-origin: *
access-control-expose-headers: Content-Disposition
content-length: 165
content-type: application/json
date: Sat,03 Oct 2026 01:57:38 GMT
server: Jetty(9.4.53.v20231009)
```



5. Consultar utilizando parámetros

- Utilice GET /pet/findByStatus y consulte las mascotas con estado available.
- Identifique el parámetro utilizado. **R/ status**
- Explique dónde se envía ese parámetro. **R/ Se envía en la URL como Query Parameter y aparece al final de la ruta base sucediendo a un ?**
- Observe la estructura de la respuesta. **R/ Lista de objetos JSON**
- Indique qué sucede si cambia el valor del parámetro por pending. **R/ La API realiza un nuevo filtrado de la base de datos y devuelve un arreglo JSON cuyo parámetro status es pending.**

Mascotas con available:

```
Code    Details
200
Response body
[
  {
    "id": 12345,
    "category": {
      "id": 1,
      "name": "dogs"
    },
    "name": "fluffy",
    "photoUrls": [],
    "tags": [
      {
        "id": 0,
        "name": "tag1"
      },
      {
        "id": 1,
        "name": "tag2"
      }
    ],
    "status": "available"
  },
  {
    "id": 123456,
    "category": {
      "id": 1,
      "name": "Dogs"
    },
    "name": "Oleh",

```

Mascotas Pending:

```
Code    Details
200
Response body
[
  {
    "id": 108,
    "category": {
      "id": 10808,
      "name": "Perros"
    },
    "name": "Kobe",
    "photoUrls": [
      "String"
    ],
    "tags": [
      {
        "id": 108,
        "name": "Doki"
      }
    ],
    "status": "pending"
  },
  {
    "id": 79481,
    "name": "pet-79481",
    "photoUrls": [],
    "tags": [],
    "status": "pending"
  },
  {
    "id": 3184,

```



6. Reproducir las operaciones en Postman

- Ahora reproduzca en Postman las operaciones realizadas en Swagger. Cree una colección llamada 'MIA - Semana 13 - API'.
- La colección debe contener, como mínimo:
- GET – Consultar mascota por ID
- POST – Crear mascota
- PUT – Modificar mascota
- GET – Buscar por status
- DELETE – Eliminar mascota
- En cada solicitud configure manualmente método, URL, parámetros, headers y body cuando corresponda. No utilice únicamente la opción de importar una colección.

GET:

The screenshot shows the Postman interface with a workspace named 'Diego Lopez's Workspace'. On the left, a collection named 'MIA - Semana 13 - API' is expanded, showing a 'GET New Request' entry. The main panel displays a 'New Request' for a GET method to the URL 'https://petstore3.swagger.io/api/v3/pet/1187525'. The 'Headers' tab is active, showing a single header 'Accept' with the value 'application/json'. The 'Body' tab is also active, showing a JSON response with a status of '200 OK', a response time of '14 ms', and a size of '537 B'. The JSON body contains the following data:

```
{
  "id": 1187525,
  "category": {
    "id": 10,
    "name": "Caninos"
  },
  "name": "Jake",
  "photoUrls": [
    "https://example.com/tocco.jpg"
  ],
  "tags": [
    {
      "id": 1,
      "name": "perro"
    }
  ]
}
```



POST:

The screenshot shows the Postman interface with a workspace named "Diego Lopez's Workspace". The left sidebar displays a collection named "MIA - Semana 13 - API" containing a "New Request" and a "login" endpoint. The main panel shows a "POST" request to "https://petstore3.swagger.io/api/v3/pet". The request body is a JSON object representing a pet named "Rocco". The response is a 200 OK status with a 19 ms response time and 543 B of data. The response body is a JSON object representing the created pet.

```
POST https://petstore3.swagger.io/api/v3/pet

{
  "id": 1187525,
  "category": {
    "id": 10,
    "name": "Caninos"
  },
  "name": "Rocco",
  "photoUrls": [
    "https://example.com/rocco.jpg"
  ],
  "tags": [
    {
      "id": 1,
      "name": "extrapada"
    }
  ]
}
```

Body: 200 OK · 19 ms · 543 B · Save Response

```
{
  "id": 1187525,
  "category": {
    "id": 10,
    "name": "Caninos"
  },
  "name": "Rocco",
  "photoUrls": [
    "https://example.com/rocco.jpg"
  ],
  "tags": [
    {
      "id": 1,
      "name": "extrapada"
    }
  ]
}
```

PUT:

The screenshot shows the Postman interface with a workspace named "Diego Lopez's Workspace". The left sidebar displays a collection named "MIA - Semana 13 - API" containing a "New Request" and a "login" endpoint. The main panel shows a "PUT" request to "https://petstore3.swagger.io/api/v3/pet". The request body is a JSON object representing a pet named "Jake". The response is a 200 OK status with a 20 ms response time and 537 B of data. The response body is a JSON object representing the updated pet.

```
PUT https://petstore3.swagger.io/api/v3/pet

{
  "id": 1187525,
  "category": {
    "id": 10,
    "name": "Caninos"
  },
  "name": "Jake",
  "photoUrls": [
    "https://example.com/rocco.jpg"
  ],
  "tags": [
    {
      "id": 1,
      "name": "extrapada"
    }
  ]
}
```

Body: 200 OK · 20 ms · 537 B · Save Response

```
{
  "id": 1187525,
  "category": {
    "id": 10,
    "name": "Caninos"
  },
  "name": "Jake",
  "photoUrls": [
    "https://example.com/rocco.jpg"
  ],
  "tags": [
    {
      "id": 1,
      "name": "extrapada"
    }
  ]
}
```



Universidad
Rafael Landívar

Identidad Jesuita en Guatemala



GET:

Diego Lopez's Workspace

POST Usuario Autenticado Co... MIA - Semana 13 - API GET New Request

MIA - Semana 13 - API New Request

GET https://petstore3.swagger.io/api/v3/pet/findByStatus?status=available

Send

Docs Params Authorization Headers 10 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
status	available			
Key	Value	Description		

Body Cookies Headers 9 Test Results

200 OK · 16 ms · 126.42 KB Save Response

{ } JSON Preview Visualize

```
8   "name": "fluffy",
9   "photoUrls": [],
10  "tags": [
11    {
12      "id": 0,
13      "name": "tag1"
14    },
15    {
16      "id": 1,
17      "name": "tag2"
18    }
19  ],
20  "status": "available"
21 }
```

Open Folder Terminal Console

Standing by for launch

Cloud Agent Globals Vault Tools

DELETE:



Diego Lopez's Workspace

+

+

+

COLLECTIONS

MIA - Semana 13 - API

GET New Request

MIA_Semana 13

GET login

POST Usuario Autenticado

POST Usuario Autenticado Copy

My Collection

ENVIRONMENTS

DOCUMENTS

SPECS

MOCKS

DATASETS

TOOLSETS

FLOWS

Search

POST Usuario Autenticado Co

MIA - Semana 13 - API

DEL New Request

No Environment

MIA - Semana 13 - API

New Request

Save

Share

DELETE

https://petstore3.swagger.io/api/v3/pet/1187525

Send

Docs

Params

Authorization

Headers 10

Body

Scripts

Settings

Cookies

Headers

9 hidden

Key	Value	Description	Bulk Edit	Presets	
✓ Accept	application/json				
Key	Value	Description			

Body

Cookies

Headers 9

Test Results

200 OK

43 ms

382 B

Save Response

Preview

Visualize

1 Pet deleted

Open Folder

Terminal

Console

Standing by for launch

Cloud Agent

Globals

Vault

Tools



7. Prueba de errores

Una API también debe analizarse cuando algo sale mal. Realice al menos tres pruebas que generen respuestas diferentes a una solicitud exitosa.

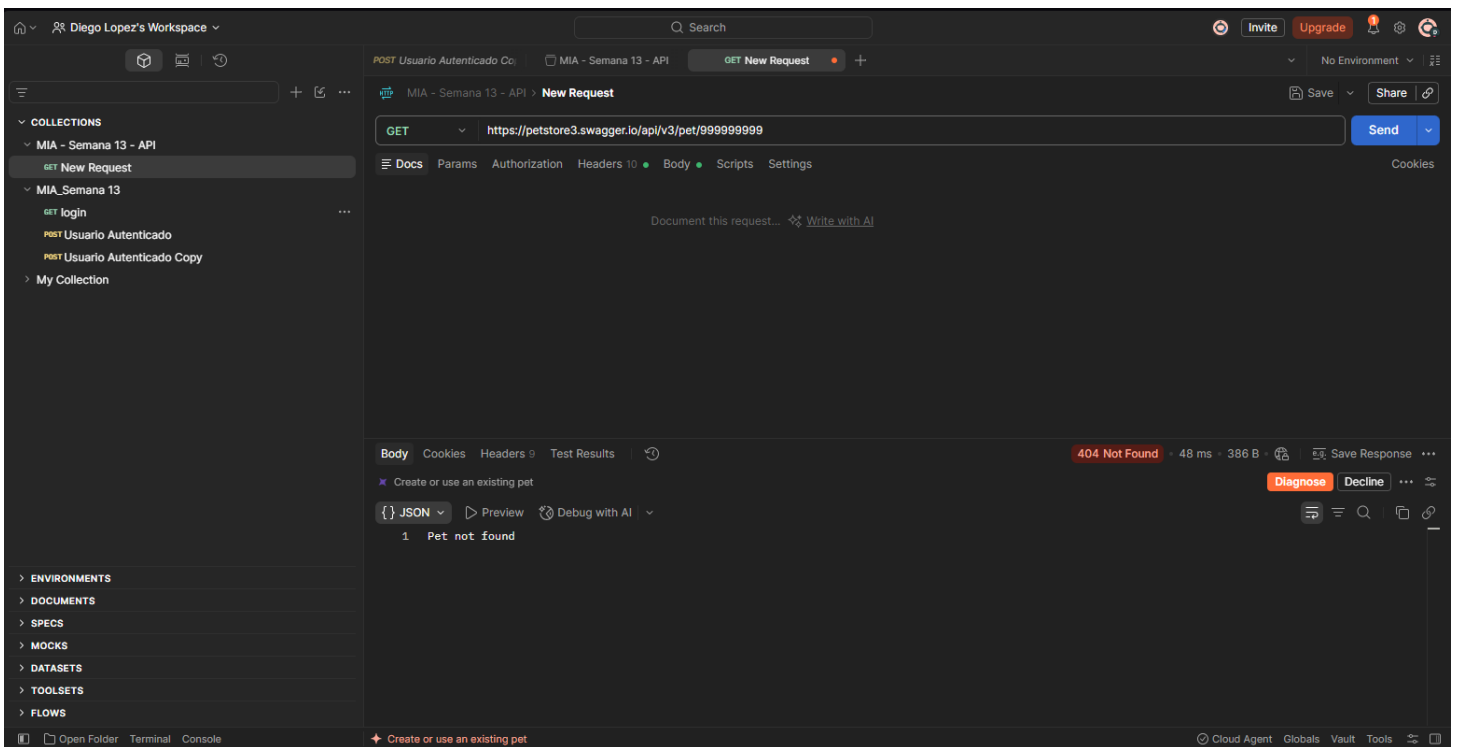
Ejemplos de escenarios:

1. Consultar un ID inexistente.
2. Enviar un JSON incorrecto o incompleto.
3. Utilizar un parámetro con un valor no esperado.
4. Intentar eliminar un recurso que ya no existe.

Para cada prueba registre:

- a) Solicitud realizada.
- b) Código HTTP recibido.
- c) Respuesta de la API.
- d) Qué esperaba que ocurriera.
- e) Qué ocurrió realmente.
- f) Explicación de la causa

1.

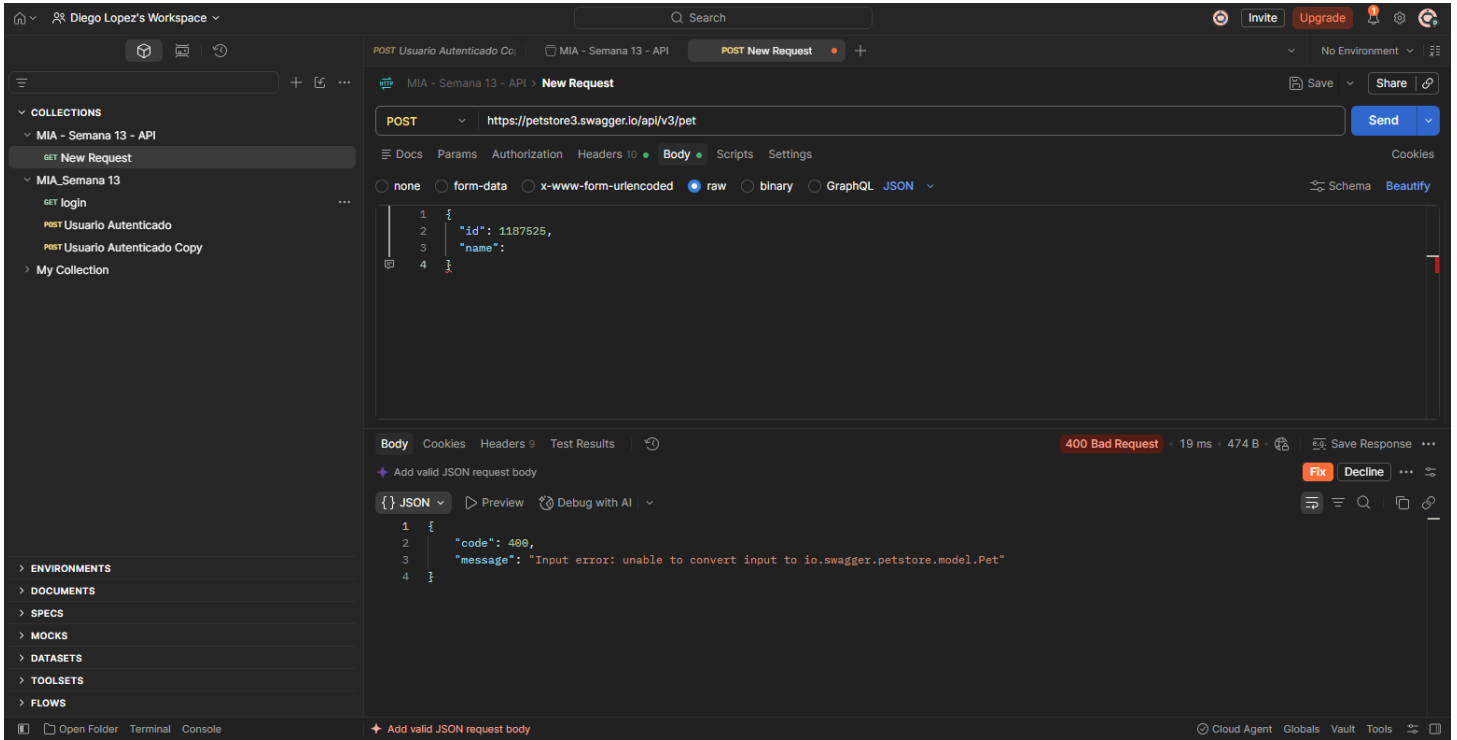


- a) Método: GET
URL: <https://petstore3.swagger.io/api/v3/pet/999999999>
- b) Código HTTP recibido: 404 not found
- c) Respuesta de la API: "Pet not found"
- d) Se esperaba que la API notificara que el recurso no existe en la base de datos



- e) El servidor respondió adecuadamente con el código 404 Not Found y un mensaje indicando que no se encontró la mascota.
- f) El identificador 999999999 no corresponde a ningún registro en la base de datos del servidor, por lo que el controlador REST devuelve la excepción de recurso no encontrado de acuerdo con las especificaciones del estándar HTTP.

2.



a) Método: POST

URL: <https://petstore3.swagger.io/api/v3/pet>

JSON:

```
{
  "id": 1187525,
  "name":
}
```

b) Código HTTP recibido: 400 Bad Request

c) Respuesta de la API:

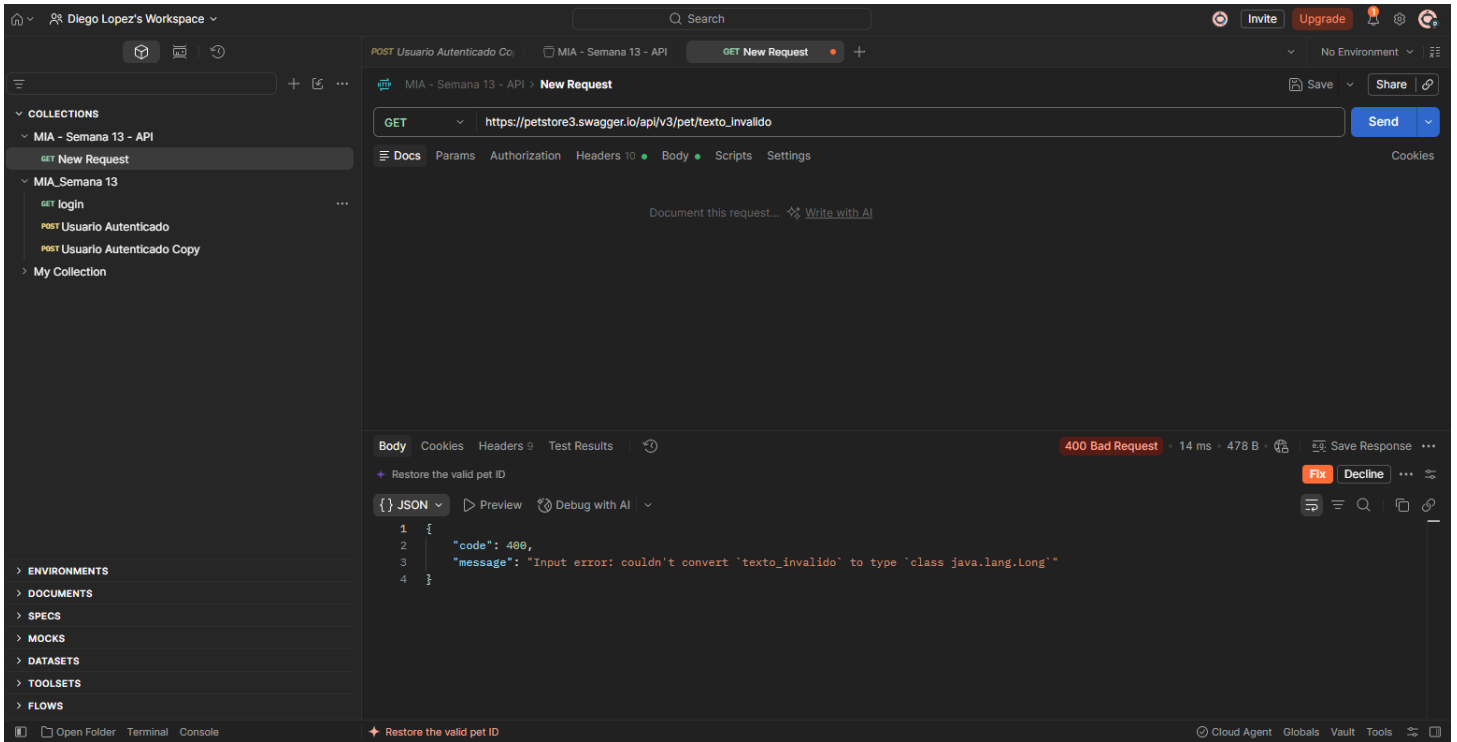
```
{
  "code": 400,
  "message": "Input error: unable to convert input to io.swagger.petstore.model.Pet"
}
```



- d) **Se esperaba que el servidor rechazara la solicitud inmediatamente al no poder interpretar la estructura del archivo enviado.**
- e) **La API devolvió un código 400 Bad Request indicando que la petición contiene una estructura incorrecta.**
- f) **El parser de JSON en el backend detectó una violación del formato estándar.**



3.

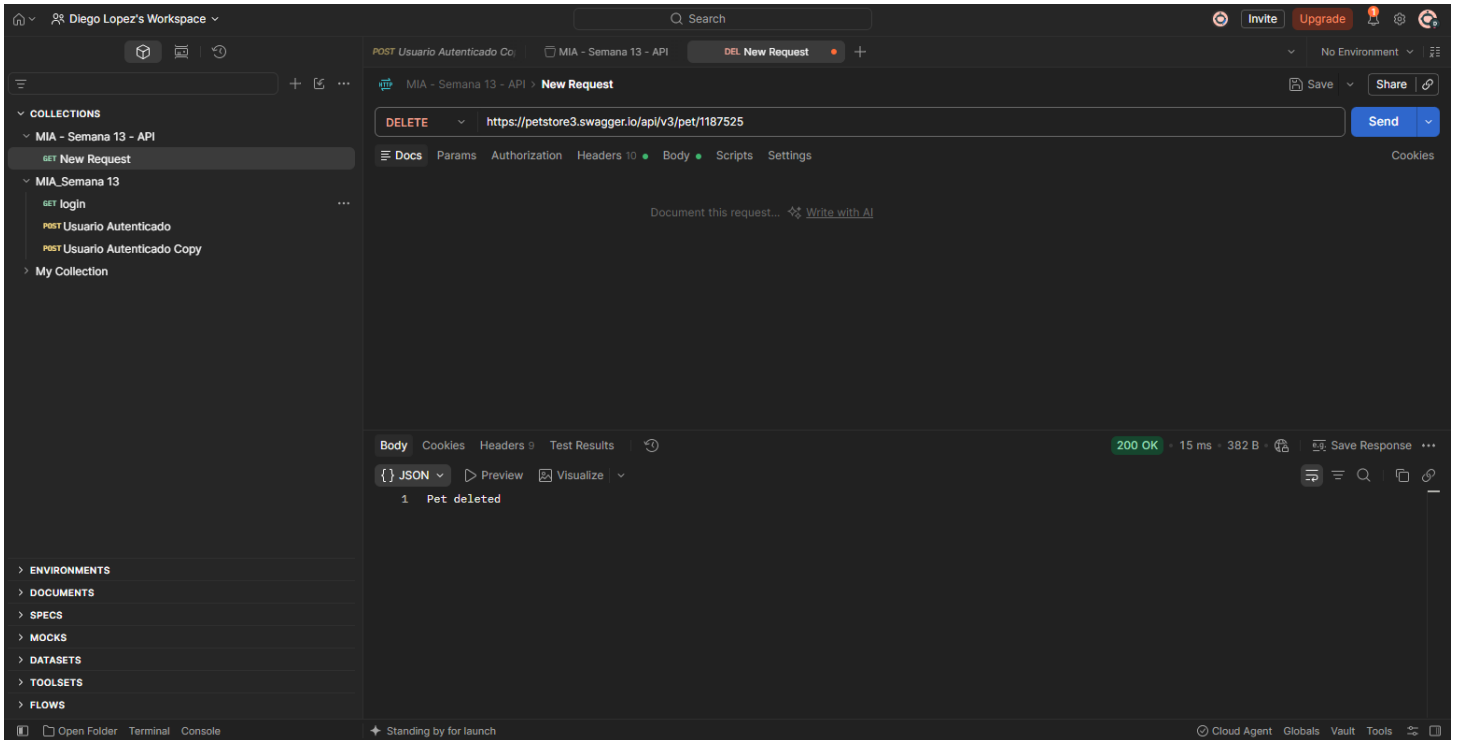


- a) Método: GET
URL: https://petstore3.swagger.io/api/v3/pet/texto_invalido
- b) Código HTTP Recibido: 400 Bad Request
- c) Respuesta de la API:

```
{  
  "code": 400,  
  "message": "Input error: couldn't convert `texto_invalido` to type `class java.lang.Long`"  
}
```
- d) Que el servidor rechazara la consulta porque el parámetro {petId} requiere obligatoriamente un número
- e) La API rechazó la solicitud devolviendo un código 400 Bad Request notificando un error en el tipo de dato.
- f) El enrutador y validador de tipos de la API analiza el segmento de la URL {petId}. Al recibir caracteres alfabéticos en lugar de un valor numérico, la conversión falla y la solicitud es interceptada antes de llegar a la base de datos.



4.



- Método: DELETE**
URL: <https://petstore3.swagger.io/api/v3/pet/1187525>
- Código HTTP recibido: 200 OK**
- Respuesta de la API: Pet deleted**
- Se esperaba recibir un código de error 404 Not Found al intentar eliminar un recurso que ya había sido borrado previamente.**
- La API devolvió un código 200 OK procesando la solicitud con éxito.**
- En arquitectura REST, algunos endpoints DELETE se diseñan con el principio de idempotencia (o bien la maqueta de prueba de Swagger no valida si el ID existe en base de datos antes de procesar el borrado). La idempotencia garantiza que ejecutar la misma solicitud varias veces deja el sistema en el mismo estado final sin fallar.**

8. API Detective

Suponga que usted recibe el siguiente requerimiento de un sistema:

“El sistema debe registrar una mascota, consultar su información, actualizar su estado y finalmente eliminarla. Si el recurso no existe, el sistema debe poder identificarlo mediante la respuesta de la API.” Resuelva el reto:

- Defina qué endpoint utilizaría para cada operación.
- Defina el método HTTP correspondiente.
- Indique qué respuesta debe analizar el programa.
- Ejecute el flujo completo en Postman.



Capture evidencia de cada paso.

Operación Requerida	Método HTTP	Endpoint	Respuesta a analizar por el programa
1. Registrar mascota	POST	https://petstore3.swagger.io/api/v3/pet	Código: 200 OK Body: JSON con el objeto creado y su id.
2. Consultar información	GET	https://petstore3.swagger.io/api/v3/pet/{petId}	Éxito (200 OK): Devuelve el JSON con la información. No existe (404 Not Found): Indica que la mascota no está registrada.
3. Actualizar estado	PUT	https://petstore3.swagger.io/api/v3/pet	Código: 200 OK Body: JSON actualizado reflejando el nuevo valor del atributo status.
4. Eliminar mascota	DELETE	https://petstore3.swagger.io/api/v3/pet/{petId}	Código: 200 OK Body: Mensaje de confirmación del borrado.
5. Identificar recurso inexistente	GET	https://petstore3.swagger.io/api/v3/pet/{petId}	Código: 404 Not Found Body: "Pet not found". Es la clave con la que el sistema confirma que el recurso ya no existe.

1.



Diego Lopez's Workspace

Search

POST Usuario Autenticado Co MIA - Semana 13 - API POST New Request

MIA - Semana 13 - API New Request

POST https://petstore3.swagger.io/api/v3/pet

Docs Params Authorization Headers 10 Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "id": 1187525,
3   "name": "Rocco",
4   "status": "available"
5 }
```

Body Cookies Headers Test Results

200 OK 15 ms 446 B Save Response

{ } JSON Preview Visualize

```
1 {
2   "id": 1187525,
3   "name": "Rocco",
4   "photoUrls": [],
5   "tags": [],
6   "status": "available"
7 }
```

Open Folder Terminal Console

Standing by for launch

Cloud Agent Globals Vault Tools



2.

The screenshot shows the Postman interface with a workspace named "Diego Lopez's Workspace". The left sidebar displays a collection of requests under "MIA - Semana 13 - API", including "New Request", "login", "Usuario Autenticado", and "Usuario Autenticado Copy". The main panel shows a "New Request" for a GET method to the URL "https://petstore3.swagger.io/api/v3/pet/1187525". The response is a JSON object with the following structure:

```
1 {
2   "id": 1187525,
3   "name": "Rocco",
4   "photoUrls": [],
5   "tags": [],
6   "status": "available"
7 }
```

The status bar indicates a successful response: "200 OK" with a response time of "88 ms" and a body size of "441 B".

3.

The screenshot shows the Postman interface with a workspace named "Diego Lopez's Workspace". The left sidebar displays a collection of requests under "MIA - Semana 13 - API", including "New Request", "login", "Usuario Autenticado", and "Usuario Autenticado Copy". The main panel shows a "New Request" for a PUT method to the URL "https://petstore3.swagger.io/api/v3/pet". The response is a JSON object with the following structure:

```
1 {
2   "id": 1187525,
3   "name": "Rocco",
4   "photoUrls": [],
5   "tags": [],
6   "status": "sold"
7 }
```

The status bar indicates a successful response: "200 OK" with a response time of "15 ms" and a body size of "441 B".



4.

The screenshot shows the Postman interface with a workspace named "Diego Lopez's Workspace". On the left, the "COLLECTIONS" panel shows a collection named "MIA - Semana 13 - API" with a sub-collection "MIA_Semana 13" containing requests like "login", "POST Usuario Autenticado", and "POST Usuario Autenticado Copy". The main panel displays a "New Request" for the endpoint `https://petstore3.swagger.io/api/v3/pet/1187525` with the method set to **DELETE**. The "Body" tab is selected, showing a raw JSON body:

```
{  "id": 1187525,  "name": "Rocco",  "status": "sold"}
```

. The response at the bottom shows a **200 OK** status with a response time of 22 ms and a body of `Pet deleted`.

5.

The screenshot shows the Postman interface with the same workspace. The main panel displays a "New Request" for the endpoint `https://petstore3.swagger.io/api/v3/pet/1187525` with the method set to **GET**. The "Body" tab is selected, showing the same raw JSON body as in the previous screenshot. The response at the bottom shows a **404 Not Found** status with a response time of 17 ms and a body of `Pet not found`. The status bar at the bottom indicates "Thinking...".